

# LedgerAI: Work Breakdown & Development Plan

**Document Status:** Active

**Project:** LedgerAI – Your Autonomous AI Finance Department

**Timeline:** 7 Days

**Team Size:** 3 Developers

This document serves as the team's single source of truth for development planning. It is designed to ensure strict separation of concerns so that all three developers can work independently, in parallel, and without blocking one another.

## Section 1 — Frontend Lead

### 1. Objective

The Frontend Lead owns the entire user-facing application. Your core responsibility is to translate the autonomous actions of the multi-agent system into an intuitive, real-time "Glass Box" experience for the user. You will not write backend business logic; you will strictly consume predefined REST APIs and Server-Sent Events (SSE).

### 2. Required Technologies

- **Next.js 14 (App Router):** Provides robust routing, server-side rendering, and optimal performance.
- **React & TypeScript:** For type-safe, component-driven UI development.
- **TailwindCSS:** For rapid, utility-first styling without context switching.
- **Zustand:** For lightweight, boilerplate-free global state management.
- **Recharts:** To render financial data and cash flow forecasts cleanly.
- **Framer Motion:** To animate the "Glass Box" agent timeline smoothly as new thoughts stream in.
- **Server-Sent Events (SSE):** Native browser API to consume real-time agent execution logs without WebSockets overhead.
- **React Hook Form & Zod:** For performant form state management and strict schema validation during Human-in-the-Loop (HITL) inputs.

### 3. Folder Ownership

You have exclusive write access to the frontend/ directory.

- frontend/app/ — Next.js routing and page layouts.
- frontend/components/ — Reusable UI building blocks.
- frontend/hooks/ — Custom React hooks (e.g., useSSE).
- frontend/store/ — Zustand state slices.
- frontend/lib/ — Utility functions, API clients (Axios/Fetch), and Zod schemas.
- frontend/styles/ — Global CSS and Tailwind configuration.

- frontend/public/ — Static assets and icons.

## 4. Pages to Build

- **Dashboard:**
  - *Purpose:* The executive summary view.
  - *Components:* KPI Cards, Cash Flow Chart (Recharts), Pending Approvals list.
  - *APIs Required:* GET /api/dashboard/summary.
  - *States:* Loading skeleton, populated data, error boundary.
- **Upload:**
  - *Purpose:* Ingest financial documents.
  - *Components:* Drag-and-drop zone, file list, upload progress bar.
  - *APIs Required:* POST /api/documents/upload.
  - *User Interactions:* File selection, drop events, clear queue.
- **Reports:**
  - *Purpose:* Display finalized accounting statements.
  - *Components:* Data tables, export to PDF button, narrative insight cards.
  - *APIs Required:* GET /api/reports.
- **Glass Box (Agent Timeline):**
  - *Purpose:* Visualize the AI's internal reasoning.
  - *Components:* Animated timeline, Agent status badges, Reasoning Cards, RAG citation links.
  - *APIs Required:* GET /api/stream/execution/{task\_id} (SSE).
  - *States:* Streaming, Paused for HITL, Completed.
- **Settings:**
  - *Purpose:* Manage company profile and tax IDs.
  - *Components:* Zod-validated forms.
  - *APIs Required:* GET /api/settings, PUT /api/settings.
- **Authentication:**
  - *Purpose:* Secure access.
  - *Components:* Login form, JWT storage logic.
  - *APIs Required:* POST /api/auth/login.
- **Loading & 404:**
  - *Purpose:* Graceful degradation and routing fallbacks.

## 5. Components

### Reusable UI Checklist:

- [ ] Navbar & Sidebar (Layout)
- [ ] File Upload Dropzone
- [ ] KPI Cards (Revenue, Burn Rate)
- [ ] Charts (Line/Bar via Recharts)
- [ ] Agent Timeline Node (Framer Motion)
- [ ] Reasoning Cards (Markdown renderer for agent thoughts)
- [ ] Status Badges (Pending, Approved, Flagged)

- [ ] HITL Approval Modal
- [ ] Data Tables (Transactions)
- [ ] Skeleton Loaders & Spinners
- [ ] Toasts / Notification popups
- [ ] Primary / Secondary Buttons

## 6. State Management

- **Global State (Zustand):** User session (JWT), active workspace, UI theme.
- **Local State (React useState):** Form inputs, modal visibility, dropdown toggles.
- **SSE State:** An array of incoming agent events appended in real-time, managed in a custom useAgentStream hook.
- **Report State:** Cached API responses for financial tables.

## 7. API Integrations

- POST /api/auth/login — Authenticates user, returns JWT.
- POST /api/documents/upload — Multipart form data upload. Returns task\_id.
- GET /api/stream/execution/{task\_id} — Connects to SSE. Yields JSON strings of agent activity.
- POST /api/hitl/approve — Submits human feedback/data to resume a paused agent graph.
- GET /api/reports/summary — Fetches dashboard metrics.

## 8. Deliverables

- [ ] Responsive Dashboard with functioning charts.
- [ ] Upload workflow with progress indicators.
- [ ] Live Glass Box page rendering SSE events sequentially.
- [ ] HITL Approval Modals integrated with graph resumption.
- [ ] Authentication UI and protected routes.

## 9. Milestones

- **Day 1:** Next.js scaffold, Tailwind config, standard component library setup.
- **Day 2:** Auth pages, global state setup, mock API integration.
- **Day 3:** Dashboard layout and Recharts implementation using mock JSON.
- **Day 4:** Document Upload UI and integration with backend upload endpoint.
- **Day 5:** Glass Box UI built and connected to live SSE endpoint.
- **Day 6:** HITL modal integration, error handling, loading states.
- **Day 7:** Final UI polish, responsiveness checks, demo rehearsal.

## 10. Definition of Done

- Application runs flawlessly in browser with zero console errors.
- All API contracts are fulfilled.
- SSE stream accurately renders without UI freezing.
- Responsive on standard desktop and tablet resolutions.

# Section 2 — Backend Lead

## 1. Objective

The Backend Lead owns the infrastructure, data persistence, and the ingestion pipeline. You are responsible for securely accepting user input, extracting raw data via OCR, persisting application state, and serving real-time events to the frontend. **Crucially, you own raw document extraction (OCR), but you pass the raw text to the AI Lead for semantic understanding.**

## 2. Required Technologies

- **Python 3.11+:** Core language.
- **FastAPI:** Asynchronous, high-performance web framework for REST and SSE.
- **SQLAlchemy 2.0 & Alembic:** ORM and migration management.
- **PostgreSQL & pgvector:** Relational data storage and vector embedding storage for the RAG pipeline.
- **Docker & Uvicorn:** Containerization and ASGI server for deployment.
- **Pydantic (V2):** Strict data validation and OpenAPI schema generation.
- **JWT (JSON Web Tokens):** Stateless authentication.

## 3. Folder Ownership

You have exclusive write access to the backend/ infrastructure directories.

- `backend/api/` — FastAPI routers and endpoints.
- `backend/database/` — SQLAlchemy setup, Alembic migrations.
- `backend/models/` — DB models (Transactions, Users, Logs).
- `backend/schemas/` — Pydantic schemas (Shared with Frontend).
- `backend/core/` — Config (Settings), security, dependencies.
- `backend/services/` — Business logic (e.g., calling OCR API).
- `backend/storage/` — S3 / local file system integration.
- `backend/auth/` — JWT generation and password hashing.

## 4. Responsibilities

- **Authentication:** Secure endpoints using OAuth2 password bearer.
- **REST APIs:** CRUD operations for users, reports, and settings.
- **Database:** Design normalized tables; ensure ACID compliance for transactions.
- **Storage & OCR Integration:** Accept PDF/Image uploads, save to blob storage, and call an OCR service (e.g., AWS Textract) to get raw text bounding boxes.
- **SSE Broadcasting:** Create an asynchronous generator endpoint that yields LangGraph state changes to the frontend.
- **Docker:** Write the docker-compose.yml to orchestrate FastAPI, Postgres, and the frontend.

## 5. APIs to Build

- POST /auth/token
  - *Input:* username, password (Form data).
  - *Output:* access\_token, token\_type.
- POST /api/documents/upload
  - *Input:* UploadFile.
  - *Output:* { "task\_id": "uuid" } (Triggers background OCR -> AI pipeline).
- GET /api/stream/execution/{task\_id}
  - *Output:* text/event-stream yielding JSON chunks of agent states.
- POST /api/hitl/resolve
  - *Input:* { "task\_id": "uuid", "provided\_data": "string" }.
  - *Output:* Status 200 OK (Unpauses LangGraph).
- GET /api/dashboard/summary
  - *Output:* Aggregated financial JSON payload.

## 6. Database Responsibilities

- **Tables:** users, companies, documents, transactions, audit\_logs, langgraph\_checkpoints.
- **Relationships:** A company has many documents and transactions. A transaction has many audit\_logs.
- **Migrations:** Managed strictly via Alembic.

## 7. Deliverables

- [ ] FastAPI app scaffolded with dependency injection.
- [ ] PostgreSQL + pgvector running in docker-compose.
- [ ] JWT authentication middleware.
- [ ] Raw OCR extraction service returning text strings.
- [ ] SSE endpoint capable of streaming async generators.
- [ ] Pydantic schemas defined and shared with Frontend Lead on Day 1.

## 8. Milestones

- **Day 1:** FastAPI + Docker + Postgres setup. Define Pydantic API contracts with Frontend.
- **Day 2:** Implement Auth, DB models, and Alembic migrations.
- **Day 3:** Build Document Upload API and integrate OCR service (raw text extraction).
- **Day 4:** Implement REST APIs for reports and dashboard metrics.
- **Day 5:** Build the SSE /stream endpoint and wrap the AI Lead's LangGraph function.
- **Day 6:** Bug fixes, logging (setup standard Python logging), and performance checks.
- **Day 7:** Cloud deployment / local demo environment finalization.

## 9. Definition of Done

- docker-compose up --build brings up the entire stack seamlessly.
- API endpoints have strict Pydantic validation.
- OCR successfully extracts raw text from test PDFs.
- Database migrations run without errors.

# Section 3 — AI / ML Lead

## 1. Objective

The AI / ML Lead owns the cognitive core of LedgerAI. You are responsible for transforming the Backend's raw OCR text into structured financial data, orchestrating the multi-agent Blackboard architecture, executing the RAG pipeline, and enforcing strict compliance (Disagree-or-Commit) via LangGraph.

## 2. Required Technologies

- **LangGraph:** For building stateful, multi-agent cyclical graphs and managing HITL pauses.
- **LangChain:** For tool calling and prompt templating.
- **OpenAI GPT-4o / GPT-4o-mini:** Foundational models. Use mini for extraction, 4o for compliance reasoning.
- **pgvector & Embeddings:** For storing and retrieving tax codes and policies.
- **Python:** Specifically focusing on AST evaluation (asteval) for the math sandbox.
- **JSON Schema:** For enforcing strictly structured LLM outputs.

## 3. Folder Ownership

You have exclusive write access to the AI logic directories within the backend.

- `backend/agents/` — Individual agent classes and LangGraph node functions.
- `backend/rag/` — Vector store initialization, chunking, and retrieval logic.
- `backend/prompts/` — System prompts and templates.
- `backend/tools/` — Python execution sandbox, Gov API mocks.
- `backend/workflows/` — The master LangGraph state machine definition.

## 4. Responsibilities & AI Pipeline

### Pipeline Flow:

Raw OCR Text (from Backend) → Partner Agent (Planning) → Accounting Agent (Entity Extraction & Ledger Draft) → Blackboard State Update → GST Agent (RAG Tax Check) → Audit Agent (Math Check & Rule Verification) → HITL (if flagged) → Partner Agent (Summary) → Database.

## 5. Agent Responsibilities

- **Partner Agent (Orchestrator):**
  - *Purpose:* Routes intent, acts as the entry/exit node of the graph.
  - *Outputs:* Next agent routing decision; final user summary.
- **Accounting Agent:**
  - *Purpose:* Semantic understanding of raw OCR text.
  - *Inputs:* Raw OCR text.
  - *Outputs:* Structured JSON (Vendor, Line items, Debit/Credit drafted).
- **GST Agent:**
  - *Purpose:* Verifies indirect tax laws using RAG.

- *Tasks:* Queries pgvector to check Section 16 of the IGST Act rules. Must explicitly flag if a transaction is a zero-rated supply to a Noida SEZ unit and verify if a Letter of Undertaking (LUT) or Bill of Export (BoE) is documented.
- **Audit Agent:**
  - *Purpose:* Enforces the Disagree-or-Commit protocol.
  - *Tasks:* Uses the Python Math Tool to recount line items. Cross-checks the GST Agent's flags. Halts the graph if an SEZ supply lacks a LUT.
- **Compliance Agent:**
  - *Purpose:* Checks historical vectors for duplicate invoices.
- **Financial Analyst Agent:**
  - *Purpose:* Takes the finalized ledger entry and generates a 2-sentence narrative on cash-flow impact.

## 6. RAG Tasks

- **Ingestion:** Create a script to chunk and embed sections of the IGST Act and SEZ compliance documents.
- **Metadata:** Tag vectors with {"type": "tax\_code", "region": "india"}.
- **Retrieval:** Implement hybrid search. Return exact citations to the GST agent.

## 7. Prompt Engineering Tasks

- Apply strict guardrails: *"You are the Audit Agent. You may NOT perform math calculations natively. You MUST use the python\_calculator tool."*
- Utilize OpenAI response\_format: {"type": "json\_object"} paired with Pydantic for guaranteed structure.

## 8. Deliverables

- [ ] LangGraph definition (StateGraph) with nodes, conditional edges, and a defined Blackboard schema.
- [ ] Working RAG retrieval chain using pgvector.
- [ ] 6 specialized agents with distinct system prompts.
- [ ] Python Math Sandbox tool.
- [ ] A main run\_workflow() function that the Backend Lead can import.

## 9. Milestones

- **Day 1:** Define the Blackboard TypedDict state. Scaffold LangChain environment.
- **Day 2:** Build Accounting Agent (Entity extraction from raw text).
- **Day 3:** Build GST Agent and implement pgvector RAG pipeline.
- **Day 4:** Build Audit Agent, Math Tool, and Disagree-or-Commit logic.
- **Day 5:** Connect all agents into LangGraph; test HITL interrupt logic.
- **Day 6:** Tune prompts to ensure 100% JSON compliance and zero math hallucinations.

- **Day 7:** Demo scenario rehearsed (specifically tuning the SEZ/LUT edge case for the pitch).

## 10. Definition of Done

- LangGraph compiles and runs end-to-end without unhandled exceptions.
- LLMs consistently return parsable JSON.
- RAG accurately retrieves relevant tax codes.
- Agent reasoning traces are clean and formatted for the Glass Box UI.

## Final Sections: Team OrchestrationFinal Deliverables Checklist

### Frontend Lead:

- ☐ Dashboard with Recharts
- ☐ Upload & Reports pages
- ☐ Live Glass Box (Agent Timeline) rendering SSE
- ☐ HITL Approval Modal
- ☐ Authentication UI

### Backend Lead:

- ☐ FastAPI application in Docker
- ☐ PostgreSQL & pgvector infrastructure
- ☐ User/Auth & REST APIs
- ☐ Document Ingestion (OCR to Raw Text)
- ☐ SSE Streaming Endpoint

### AI/ML Lead:

- ☐ LangGraph State Machine (Blackboard)
- ☐ Partner, Accounting, GST, Audit, Compliance, Analyst Agents
- ☐ RAG Pipeline (Tax Codes & SEZ logic)
- ☐ Python Tool Sandbox
- ☐ HITL state interruption and resumption

### Team Integration:

- ☐ Flawless End-to-End Workflow Execution
- ☐ No Merge Conflicts in develop
- ☐ Working Pitch Demo Script

**REMINDER:**



**CREATE YOUR OWN FORK OF THE REPOSITORY AND PUSH ONLY IN YOUR FORK. TO REFLECT THE CHANGES IN THE MAIN BRANCH YOU CAN OPEN A PR AND MERGE IF NO CONFLICTS. IF CONFLICTS PRESENT, DO NOT FORCE PUSH AND HOLD A MEET WITH THE AFFECTED MEMBER**